SynapseMatrix 技术规格书 V1.0

> **项目定位：** 端侧优先（Edge-first）、P2P 协同、隐私保护的个人分布式认知计算系统
> **项目目标：** 用一个长期项目倒逼补齐 C++、操作系统、计算机网络、数据结构与算法、数据库、AI Engineering、全栈、分布式系统、GPU/推理、安全工程等能力。
> **预计周期：** 10～12 个月
> **适用方式：** 本文同时作为「项目设计文档 + 学习总纲 + 施工图 + Milestone 验收标准」

---

0. 文档说明

0.1 这份文档解决什么问题

SynapseMatrix 不是一个“堆技术名词”的毕业项目，而是一条长期工程主线：

```text
理论
  ↓
最小实验
  ↓
项目实现
  ↓
测试
  ↓
Benchmark
  ↓
文档
  ↓
Git Commit
  ↓
下一阶段
```

每个阶段都必须同时产出：

1. 可运行代码
2. 可验证测试
3. Benchmark / 指标
4. 技术文档
5. Git 提交记录

0.2 项目核心原则

- **先本地，后分布式**
- **先正确，后性能**

- **先可观测，后优化**
- **先协议，再自动化**
- **先威胁模型，再安全承诺**
- **先手写最小实现，再引入成熟框架**
- **不以“功能数量”衡量进度，以 Milestone 验收衡量进度**
- **不追求“全球第一个”之类的宣传，而追求可解释、可复现、可 Benchmark**

---

1. 系统目标

1.1 总目标

构建一个由 PC、手机、平板、嵌入式设备等组成的个人计算网络，使多个设备可以：

- 共享计算资源
- 协同执行 AI 推理任务
- 构建统一但可控的个人记忆空间
- 在设备间增量同步认知状态
- 根据事件和上下文自动触发任务
- 在无云或弱联网场景下保持基础能力

系统最终应具备：

```text
Personal Data
      ↓
Memory Ingestion
      ↓
Cognitive Memory
      ↓
Retrieval / Reasoning
      ↓
Distributed Inference
      ↓
Task Execution
      ↓
Result
      ↓
Memory Update
```

1.2 核心用户场景

场景 A：个人知识记忆

用户导入：

- ＰＤＦ
- Ｍａｒｋｄｏｗｎ
- Ｗｏｒｄ
- 图片
- 截图
- 网页内容
- 聊天上下文
- 音频转写

系统自动形成：

```text
文档
  ↓
解析
  ↓
Ｅｍｂｅｄｄｉｎｇ
  ↓
实体
  ↓
关系
  ↓
时间
  ↓
空间
  ↓
Ｍｅｍｏｒｙ　Ｎｏｄｅ
```

场景　Ｂ：跨时间认知检索

用户问：

> “我之前为什么决定学　Ａｇｅｎｔ？”

系统不只是做向量相似度搜索，还会综合：

- 语义相似度
- 时间
- 访问历史
- 重要度
- 相关实体

- 图关系

最终构建上下文。

场景 C：多设备协同推理

PC 有强 GPU，手机有弱 GPU / NPU，嵌入式节点只有 CPU：

```text
                    Scheduler
                        |
          ┌─────────────┼─────────────┐
          ↓             ↓             ↓
        PC          Phone          Edge
   Layer 0~N     Layer N~M      CPU Task
```

系统根据：

- GPU/CPU
- VRAM/RAM
- 网络带宽
- RTT
- 电量
- 温度
- 节点负载

动态决定执行方式。

场景 D：上下文触发任务

例如：

```text
打开论文 PDF
      ↓
DocumentOpened
      ↓
识别主题
      ↓
查询相关 Memory
      ↓
发现过去研究记录
      ↓
生成任务
```

```
          ↓
总结  +  关联
          ↓
写回  Memory
```

---

2. 非目标（Non-Goals）

V1.0 明确不做以下内容：

2.1 不追求商业级云规模

不目标：

- 万级节点
- 全球跨区域集群
- 云端 SaaS
- 高并发商用平台

优先目标：

> 1～5 台个人设备的可靠协同。

2.2 不承诺“零泄露”

不使用：

- “绝对隐私”
- “数学意义上零泄露”
- “任何攻击都无法获取信息”

正式定位：

> **Privacy-by-Design / Privacy-aware Architecture**

2.3 不在早期实现完整 Federated Learning

LoRA / FedAvg / 联邦训练属于后期研究扩展。

V1.0 核心首先实现：

> **Federated Memory / Incremental Memory Synchronization**

2.4 不在一开始支持所有平台

优先：

1. Windows PC
2. Linux PC
3. Android
4. Raspberry Pi / Embedded Linux

2.5 不第一天自己实现 QUIC

学习阶段允许：

- libp2p
- MsQuic
- 其他成熟实现

真正自研的是：

> **Synapse Application Protocol + Scheduler + Memory Sync + Runtime**

---

3. 产品边界

```text
┌──────────────────────────────────────────────┐
│              Synapse Control Center          │
│  Vue3 + TypeScript + Visualization           │
├──────────────────────────────────────────────┤
│              Control Plane                   │
│  NestJS + Redis + PostgreSQL                 │
├──────────────────────────────────────────────┤
│               AI Plane                       │
│  FastAPI + Python + Model / Memory           │
├──────────────────────────────────────────────┤
│              Memory Fabric                   │
│  Neo4j + Vector + Metadata + Object Storage  │
├──────────────────────────────────────────────┤
│           Distributed Runtime                │
│  C++ + llama.cpp + Scheduler + QUIC/P2P      │
├──────────────────────────────────────────────┤
│              Device Nodes                    │
│  PC / Phone / Embedded                       │
└──────────────────────────────────────────────┘
```

```

---

# 4. 总体架构

## 4.1 八层技术架构

| Layer | 名称 | 核心职责 |
|---|---|---|
| L1 | Device Runtime | 节点执行、线程、资源 |
| L2 | Inference Engine | 模型加载与推理 |
| L3 | Distributed Inference | 多节点协同推理 |
| L4 | Memory Fabric | 个人记忆 |
| L5 | AI Pipeline | 文档/多模态处理 |
| L6 | Control Plane | 设备、任务、权限、元数据 |
| L7 | Event Runtime | 事件、触发、任务流 |
| L8 | Console | Web 控制台与可视化 |

---

# 5. L1 Device Runtime

## 5.1 技术栈

```text
C++20
CMake
vcpkg / Conan
CUDA
OpenMP
std::thread
std::atomic
std::mutex
Condition Variable
Thread Pool
```

5.2 核心模块

```text
runtime/core/
├── device/
├── scheduler/
├── worker/
```
```

```
├── executor/
├── resource/
├── memory/
└── telemetry/
```

5.3 Device Node

每个设备维护：

```text
device_id
device_name
os
cpu
gpu
ram
vram
network
battery
temperature
capabilities
status
last_heartbeat
```

5.4 Node Status

```text
DISCOVERING
     ↓
CONNECTING
     ↓
AUTHENTICATING
     ↓
READY
     ↓
BUSY
     ↓
DEGRADED
     ↓
OFFLINE
```

允许：

```text
OFFLINE → DISCOVERING
DEGRADED → READY
BUSY → READY
READY → OFFLINE
```

验收

- 能创建多个虚拟 Device
- 能获取 CPU/RAM/GPU 等基础信息
- 能发送 heartbeat
- 能检测 offline
- 能记录状态变化

---

6. L2 Inference Engine

6.1 技术栈

```text
llama.cpp
ggml
GGUF
CUDA
CUDA Runtime
```

6.2 学习目标

必须理解：

- Transformer
- Attention
- KV Cache
- Quantization
- Tensor
- Operator
- Computation Graph
- Memory Bandwidth
- GPU Kernel
- CPU / GPU 协作
- Prefill / Decode

6.3 目标能力

```text
Load Model
    ↓
Tokenizer
    ↓
Input
    ↓
Prefill
    ↓
Decode
    ↓
Sampling
    ↓
Output
```

验收

至少能够解释：

> “一次 token 生成从输入到输出发生了什么。”

并完成：

- 本地模型加载
- 单轮推理
- 流式生成
- token/s 统计
- TTFT 统计
- GPU/RAM 统计

---

7. L3 Distributed Inference

7.1 网络技术栈

首选：

```text
QUIC
HTTP/3
```

```
libp2p
TLS 1.3
```

底层学习：

```text
UDP
TCP
TLS
Congestion Control
Multiplexing
NAT
NAT Traversal
Hole Punching
Connection Migration
```

7.2 Synapse Protocol

协议版本：

```text
SYNAPSE/1.0
```

7.3 消息类型

```text
NODE_HELLO
NODE_CAPABILITY
HEARTBEAT
TASK_SUBMIT
TASK_ASSIGN
TASK_CANCEL
TASK_RESULT
TASK_ERROR
INFERENCE_START
INFERENCE_CHUNK
INFERENCE_RESULT
MEMORY_PUSH
MEMORY_PULL
MEMORY_DIFF
SYNC_ACK
```

7.4 Task Message

```json
{
  "type": "INFERENCE_TASK",
  "task_id": "task_xxx",
  "model_id": "model_xxx",
  "partition": {
    "start_layer": 12,
    "end_layer": 20
  },
  "tensor_format": "fp16",
  "deadline_ms": 200
}
```

7.5 节点发现

优先级：

```text
Local Discovery
      ↓
LAN Discovery
      ↓
Known Peers
      ↓
Optional NAT Traversal
```

7.6 模型分片

第一阶段：

```text
Model
├── Layer 0~10   → Node A
└── Layer 11~20  → Node B
```

第二阶段：

```text
Node A
```

```
   ↓
Node B
   ↓
Node C
```

强制实验

必须做：

```text
Local Inference
vs
2-Node Inference
```

测试：

- TTFT
- Token/s
- RTT
- Bandwidth
- Tensor Transfer Size
- GPU Utilization
- CPU Utilization

---

8. L3 Scheduler：推理调度器

8.1 输入

```text
CPU Score
GPU Score
VRAM Available
RAM Available
Network RTT
Bandwidth
Battery
Temperature
Current Load
Model Compatibility
```

8.2 Node Score

V1 使用可解释的加权公式：

```text
Score =
w1 * ComputeCapacity
+ w2 * AvailableMemory
+ w3 * Bandwidth
- w4 * Latency
- w5 * Load
- w6 * BatteryPenalty
- w7 * ThermalPenalty
```

权重后续通过 Benchmark 调优。

8.3 Scheduler Pipeline

```text
Request
  ↓
Discover Nodes
  ↓
Filter Capabilities
  ↓
Estimate Cost
  ↓
Generate Candidate Plans
  ↓
Score Plans
  ↓
Select Plan
  ↓
Assign Tasks
  ↓
Monitor
  ↓
Recover / Retry
```

8.4 调度策略

V1：

```text
Static
```

V2：

```text
Heuristic Dynamic Scheduling
```

V3：

```text
Cost-aware Scheduling
```

V4：

```text
Adaptive Scheduling
```

验收

至少支持：

- 本地执行
- 双节点执行
- 节点失联检测
- 任务取消
- 超时重试
- fallback 到本地执行

---

9. L4 Memory Fabric

9.1 核心思想

Synapse Memory 不只是 Vector Database。

每个记忆同时拥有：

```text
Semantic
```

```
Temporal
Spatial
Relational
Importance
Activation
Source
Version
```

9.2 Memory Node 数据结构

```json
{
  "memory_id": "mem_xxx",
  "content": "...",
  "embedding": [],
  "summary": "...",
  "source": {
    "type": "document",
    "id": "doc_xxx"
  },
  "created_at": "2026-01-01T10:00:00Z",
  "updated_at": "2026-01-01T10:00:00Z",
  "last_accessed_at": "2026-01-03T10:00:00Z",
  "location": {
    "lat": 0,
    "lon": 0
  },
  "importance": 0.8,
  "activation": 0.63,
  "access_count": 12,
  "version": 3,
  "hash": "...",
  "entities": [],
  "relations": []
}
```

9.3 Memory 类型

```text
DOCUMENT
NOTE
CHAT
IMAGE
```

```
SCREENSHOT
AUDIO
VIDEO
EVENT
LOCATION
TASK
DECISION
PREFERENCE
PROJECT
PERSON
CONCEPT
```

9.4 Memory Score

初始模型：

```text
MemoryScore =
a * SemanticSimilarity
+ b * Recency
+ c * Importance
+ d * Activation
+ e * GraphRelevance
```

参数不在 V1 固定为“科学真理”，必须通过 Benchmark 调整。

9.5 激活与衰减

初版：

```text
activation(t)
      =
      activation0 * exp(-λ * Δt)
```

重新访问：

```text
activation_new
=
min(1.0, activation_old + reinforcement)
```

后续研究：

- spaced repetition
- Hebbian-inspired reinforcement
- graph-aware activation
- context-sensitive activation

---

10. L4 图谱设计

10.1 Graph Entity

```text
Person
Project
Topic
Document
Task
Organization
Location
Event
Technology
Decision
```

10.2 Relation

```text
RELATED_TO
MENTIONS
AUTHORED_BY
PART_OF
LOCATED_AT
HAPPENED_AT
DERIVED_FROM
CONTRADICTS
SUPPORTS
CAUSED_BY
USED_IN
```

10.3 时间关系

```text
created_at
updated_at
valid_from
valid_to
last_accessed_at
```

10.4 空间关系

支持：

```text
lat
lon
place_id
geohash
```

V1 可以只保留可选字段，不强制采集精确位置。

---

11. L5 AI Pipeline

11.1 技术栈

```text
Python
FastAPI
PyTorch
Transformers
Sentence Transformers
ONNX Runtime
```

11.2 文档

```text
PyMuPDF
python-docx
openpyxl
```

11.3 多模态

```text
OCR
ASR
Image Embedding
Document Parsing
```

候选技术：

```text
PaddleOCR
Whisper
CLIP / SigLIP 类模型
```

11.4 Pipeline

```text
Input
  ↓
Parser
  ↓
Normalizer
  ↓
Chunker
  ↓
Embedding
  ↓
Entity Extraction
  ↓
Relation Extraction
  ↓
Metadata
  ↓
Memory Builder
  ↓
Graph + Vector
```

验收

导入任意一份 PDF 后，系统能够：

- 提取文本

- 切分
- 生成 embedding
- 抽取实体
- 生成基础关系
- 建立 memory node
- 支持检索

---

12. L6 Control Plane

12.1 技术栈

```text
NestJS
TypeScript
PostgreSQL
Redis
```

12.2 模块

```text
control-plane/
├── auth/
├── users/
├── devices/
├── models/
├── tasks/
├── memory/
├── sync/
├── events/
├── policies/
└── telemetry/
```

12.3 PostgreSQL 核心表

users

```text
id
username
created_at
updated_at
```

```

### devices

```text
id
device_id
name
platform
status
public_key
last_seen_at
created_at
updated_at
```

models

```text
id
name
version
format
size
quantization
created_at
```

tasks

```text
id
type
status
priority
created_at
started_at
finished_at
assigned_node
retry_count
error
```

events
```

```text
id
type
source
payload
created_at
```

memory_metadata

```text
memory_id
source_type
source_id
version
hash
created_at
updated_at
```

---

13. L7 Event Runtime

13.1 事件总线

第一版：

```text
Redis Streams
```

后续可研究：

```text
Kafka
NATS
```

13.2 事件类型

```text
DEVICE_ONLINE
DEVICE_OFFLINE
DOCUMENT_CREATED
```

```
DOCUMENT_OPENED
MEMORY_CREATED
MEMORY_UPDATED
MODEL_LOADED
INFERENCE_STARTED
INFERENCE_FINISHED
SYNC_STARTED
SYNC_COMPLETED
TASK_FAILED
```

13.3 Cognitive Event Runtime

```text
Event
  ↓
Context Aggregation
  ↓
Semantic Classification
  ↓
Policy Evaluation
  ↓
Planner
  ↓
Task Graph
  ↓
Execution
  ↓
Observation
  ↓
Memory Update
```

---

14. L8 Web Console

14.1 技术栈

```text
Vue 3
TypeScript
Vite
Pinia
Vue Router
```

```
Tailwind CSS
```

可视化：

```text
Three.js
WebGL
D3.js
ECharts
```

14.2 页面

```text
Dashboard
Devices
Inference
Memory
Graph
Timeline
Tasks
Workflow
Logs
Settings
Security
Benchmarks
```

14.3 Dashboard

显示：

```text
Nodes
Models
Memory Count
Active Tasks
Token/s
GPU Utilization
Network RTT
Sync State
```

14.4 Memory Explorer

支持：

```text
Timeline
Topic
Entity
Location
Graph
Semantic Search
```

---

15. 安全架构

15.1 身份

每台设备拥有：

```text
Device ID
Public Key
Private Key
Certificate
```

15.2 通信安全

```text
Mutual Authentication
+
TLS 1.3
+
QUIC
```

15.3 权限

使用 Capability-based 思路：

```text
Device A
   ├── READ_MEMORY
   ├── RUN_INFERENCE
```

```
    └── WRITE_MEMORY
```

不同设备拥有不同权限。

15.4 威胁模型

至少考虑：

```text
MITM
Replay
Unauthorized Device
Credential Theft
Message Tampering
Malicious Node
Stolen Device
Memory Leakage
```

15.5 安全原则

- 原始数据尽量本地处理
- 明确同步边界
- 传输加密
- 最小权限
- 可吊销设备
- 操作可审计

---

16. Memory Synchronization

16.1 目标

设备之间不默认同步全部原始内容，而是根据策略同步：

```text
Memory ID
Version
Metadata
Embedding
Hash
Relations
Activation
```

```
Importance
```

原始对象可按需请求。

16.2 Version

每个 memory：

```text
memory_id
version
hash
updated_at
```

16.3 Diff Sync

```text
Device A: v17
Device B: v12

        ↓

只传输 v13~v17 的变化
```

16.4 冲突

V1：

```text
Last Write Wins
```

V2：

```text
Version Vector
```

V3：

```text
CRDT / Semantic Merge
```

```

## 16.5 同步状态机

```text
IDLE
  ↓
DISCOVER
  ↓
COMPARE_VERSION
  ↓
REQUEST_DIFF
  ↓
VERIFY
  ↓
APPLY
  ↓
ACK
  ↓
DONE
```

失败：

```text
RETRY
BACKOFF
DEGRADED
```

---

17. API 设计

17.1 Control Plane

Device

```http
POST /api/v1/devices/register
GET  /api/v1/devices
GET  /api/v1/devices/:id
POST /api/v1/devices/:id/heartbeat
POST /api/v1/devices/:id/revoke
```
```

Models

```http
GET  /api/v1/models
POST /api/v1/models
GET  /api/v1/models/:id
POST /api/v1/models/:id/load
POST /api/v1/models/:id/unload
```

Tasks

```http
POST /api/v1/tasks
GET  /api/v1/tasks
GET  /api/v1/tasks/:id
POST /api/v1/tasks/:id/cancel
```

Memory

```http
POST /api/v1/memory
GET  /api/v1/memory/:id
POST /api/v1/memory/search
POST /api/v1/memory/retrieve
POST /api/v1/memory/:id/reinforce
```

Sync

```http
POST /api/v1/sync/start
GET  /api/v1/sync/status
POST /api/v1/sync/resolve
```

---

18. Agent API

```http
POST /api/v1/agent/chat
POST /api/v1/agent/stream
```

```
POST /api/v1/agent/plan
POST /api/v1/agent/execute
```

Agent Pipeline：

```text
User Query
  ↓
Intent
  ↓
Memory Retrieval
  ↓
Graph Expansion
  ↓
Context Construction
  ↓
Planner
  ↓
Tool Selection
  ↓
Task Execution
  ↓
Answer
  ↓
Memory Update
```

---

19. 项目目录结构

```text
synapse-matrix/
│
├── apps/
│   ├── console/
│   ├── control-plane/
│   └── ai-service/
│
├── runtime/
│   ├── core/
│   │   ├── device/
│   │   ├── scheduler/
│   │   ├── executor/
```

```
│   │   ├── worker/
│   │   └── telemetry/
│   │
│   ├── inference/
│   │   ├── llama/
│   │   ├── ggml/
│   │   └── partition/
│   │
│   ├── network/
│   │   ├── discovery/
│   │   ├── transport/
│   │   ├── protocol/
│   │   └── p2p/
│   │
│   └── security/
│       ├── identity/
│       ├── crypto/
│       └── certificate/
│
├── memory/
│   ├── ingestion/
│   ├── parser/
│   ├── embedding/
│   ├── extraction/
│   ├── graph/
│   ├── retrieval/
│   ├── activation/
│   └── sync/
│
├── agents/
│   ├── planner/
│   ├── executor/
│   ├── tools/
│   └── event-runtime/
│
├── protocols/
│   └── synapse-protocol/
│
├── benchmarks/
│   ├── inference/
│   ├── memory/
│   ├── network/
│   └── scheduler/
│
├── tests/
```

```
│      ├── unit/
│      ├── integration/
│      ├── e2e/
│      └── chaos/
│
├── docs/
│      ├── architecture/
│      ├── protocol/
│      ├── research/
│      └── benchmark/
│
├── deploy/
│      ├── docker/
│      └── compose/
│
├── .github/
│      └── workflows/
│
└── README.md
```

---

20. Benchmark 体系

20.1 Inference Benchmark

指标：

```text
TTFT
Token/s
Latency
VRAM
RAM
GPU Utilization
Network Bytes
```

对比：

```text
1 Node
2 Nodes
3 Nodes
```

```

## 20.2 Memory Benchmark

对比：

```text
Naive RAG
Vector RAG
Graph RAG
Synapse Memory
```

指标：

```text
Recall
Precision
Retrieval Latency
Context Relevance
```

20.3 Scheduler Benchmark

对比：

```text
Static Scheduling
Round Robin
Heuristic Scheduling
Synapse Scheduler
```

指标：

```text
Throughput
P95 Latency
Resource Utilization
Task Completion Rate
Failure Recovery Time
```

20.4 Sync Benchmark
```

测试：

```text
10 Memories
100 Memories
1,000 Memories
10,000 Memories
```

指标：

```text
Sync Time
Transferred Bytes
Conflict Count
Recovery Time
Consistency
```

20.5 Memory Decay Benchmark

观察：

```text
Day 1
Day 7
Day 30
Day 90
```

指标：

```text
Activation
Retrieval Probability
Reinforcement Count
```

---

21. 可观测性

必须记录：

```text
```

```
Logs
Metrics
Traces
```

推荐：

```text
OpenTelemetry
Prometheus
Grafana
```

核心指标：

```text
node_online_count
task_queue_length
task_failure_rate
inference_tokens_per_second
inference_ttft
network_rtt
network_bytes
memory_count
memory_retrieval_latency
sync_success_rate
sync_conflict_count
```

---

22. 测试策略

22.1 Unit Test

测试：

- Scheduler
- Memory Score
- Decay
- Protocol Parser
- Versioning
- Graph Operations

22.2 Integration Test

测试：

```text
NestJS
+
FastAPI
+
Neo4j
+
Redis
+
PostgreSQL
```

22.3 E2E

完整流程：

```text
Upload PDF
  ↓
Parse
  ↓
Memory
  ↓
Retrieve
  ↓
Agent
  ↓
Inference
  ↓
Answer
```

22.4 Chaos Test

人为模拟：

```text
Node Offline
Network Delay
Packet Loss
Task Timeout
Memory Conflict
```

```
Process Crash
```

---

23. CI/CD

技术：

```text
GitHub Actions
Docker
Docker Compose
CMake Build
Python Tests
TypeScript Tests
Integration Tests
```

Pull Request 至少运行：

```text
Lint
Unit Test
Build
Integration Test
```

---

24. 12 个月学习路线

Month 1 — Local Cognitive Core

必学

```text
Python
FastAPI
PostgreSQL
Neo4j
Vector Search
RAG
```

Embeddings
Document Parsing
```

### 项目

```text
PDF
 ↓
Parser
 ↓
Embedding
 ↓
Graph + Vector
 ↓
LLM
```

验收

- PDF 导入成功
- Embedding 成功
- Graph 成功
- Vector Retrieval 成功
- AI 能引用记忆

LeetCode

```text
每周 4～5 题
```

重点：

```text
Array
String
Hash
Two Pointer
Stack
Queue
Linked List
```

```

Month 2 — Cognitive Memory

必学

```text
Graph Theory
Similarity Search
Ranking
Time Decay
Information Retrieval
```

项目

实现：

```text
Importance
Activation
Decay
Graph Relevance
Temporal Retrieval
```

验收

给定相同问题：

```text
Vector RAG
vs
Synapse Memory
```

能够做 Benchmark。

LeetCode

```text
每周 4 题
```

新增：

```text
Binary Search
Tree
DFS
BFS
```

Month 3 — C++ Runtime

必学

```text
C++20
CMake
RAII
Smart Pointer
Move Semantics
Thread
Mutex
Atomic
Condition Variable
Thread Pool
Memory
```

项目

实现：

```text
Device
Worker
Task
Queue
Executor
Telemetry
```

验收

自己实现：

> 多线程 Task Runtime。

LeetCode

```text
每周 3～4 题
```

统一使用：

> C++

Month 4 — Network & P2P

必学

```text
OSI
TCP
UDP
Socket
TLS
QUIC
HTTP/3
NAT
NAT Traversal
```

项目

实现：

```text
Node A
  ↕
Node B
```

支持：

```text
Discovery
Connect
Auth
```

```
Heartbeat
Message
Reconnect
```

验收

两个节点可以可靠通信。

LeetCode

维持：

```text
每周 3 题
```

Month 5 — Model Runtime

必学

```text
Transformer
Attention
KV Cache
Quantization
GGUF
ggml
llama.cpp
CUDA Basics
```

项目

做到：

```text
Local Model
  ↓
Runtime
  ↓
Metrics
```

验收

能够解释：

> Prefill、Decode、KV Cache、Quantization 分别是什么。

并测得：

```text
TTFT
Token/s
VRAM
```

Month 6 — Distributed Inference

必学

```text
Model Parallelism
Pipeline Parallelism
Tensor Parallelism
Communication Cost
Serialization
Binary Protocol
```

项目

```text
Layer 0~N
    ↓
Node A

Layer N~M
    ↓
Node B
```

验收

完成：

```text
1 Node
vs
2 Node
```

Benchmark。

Month 7 — Scheduler

必学

```text
Scheduling
Load Balancing
Priority Queue
Resource Allocation
Failure Detection
```

项目

实现：

```text
Node Score
Task Priority
Dynamic Scheduling
Retry
Fallback
```

验收

节点负载变化时，Scheduler 能改变任务分配。

LeetCode

重点：

```text
Heap
```

```
Priority Queue
Graph
Topological Sort
Shortest Path
```

Month 8 — Federated Memory

必学

```text
Distributed State
Version
Consistency
Conflict
Replication
Eventual Consistency
```

项目

实现：

```text
Full Sync
  ↓
Incremental Sync
  ↓
Versioned Sync
```

验收

两台设备可以：

- 同步
- 增量同步
- 检测冲突
- 恢复同步

Month 9 — Distributed Consistency + Security

必学

```text
Version Vector
CRDT
TLS
mTLS
Public Key Cryptography
Certificate
Capability Security
Threat Modeling
```

项目

实现：

```text
Device Identity
Certificate
Permission
Memory Sync Security
Revocation
```

验收

能够模拟：

```text
Unauthorized Device
MITM Attempt
Replay
Revoked Device
```

并给出测试结果。

Month 10 — Cognitive Event Runtime

必学

```text
Event-driven Architecture
Event Bus
Task DAG
Planner
Tool Calling
Agent Runtime
```

项目

实现：

```text
Event
  ↓
Context
  ↓
Planner
  ↓
Task DAG
  ↓
Executor
  ↓
Memory
```

验收

至少实现 3 个真正的上下文触发任务。

Month 11 — Web Console

必学

```text
Vue3
TypeScript
WebSocket
Three.js
D3
ECharts
```

项目

完成：

```text
Dashboard
Device Graph
Memory Graph
Timeline
Task Monitor
Benchmark
```

验收

所有核心状态可视化。

Month 12 — Reliability + Final Release

必学

```text
Distributed Failure
Observability
Tracing
Metrics
Chaos Engineering
CI/CD
Docker
System Design
```

项目

实现：

```text
Node Failure
Network Failure
Task Failure
Recovery
Retry
```

Checkpoint
Observability
```

### 验收

完成：

```text
SynapseMatrix v1.0
```

---

25. 每日学习制度

不要再用：

> “今天看哪个视频？”

改成：

```text
今日 Milestone
  ↓
知识缺口
  ↓
理论学习
  ↓
最小实验
  ↓
接入项目
  ↓
测试
  ↓
记录
```

每天 4～6 小时推荐

项目	时间
SynapseMatrix	2.5～3.5h
CS / 408	1～1.5h
LeetCode	30～45min
```

| 总结 / 文档 | 15～30min |

---

26. LeetCode 策略

目标

不是刷题数量，而是：

```text
Data Structure
+
Algorithm
+
C++
```

基础阶段

每周：

```text
4～5 题
```

项目重阶段

每周：

```text
3～4 题
```

求职笔试阶段

根据岗位暂时提高：

```text
每天 1～2 题
```

推荐顺序

```text
```

```
Array
String
Hash
Two Pointer
Stack
Queue
Linked List
Binary Search
Tree
DFS
BFS
Heap
Graph
Topological Sort
Shortest Path
Dynamic Programming
```

核心原则

LeetCode 使用：

```text
C++
```

并且主动把算法知识映射到项目：

```text
Heap
→ Scheduler

Graph
→ Knowledge Graph

BFS / DFS
→ Graph Retrieval

LRU
→ Memory Cache

Priority Queue
→ Task Queue

Topological Sort
```

→ Task DAG
```

---

# 27. 408 学习映射

SynapseMatrix 可以直接作为 408 的实践场。

## 数据结构

```text
Array
Linked List
Tree
Graph
Heap
Hash
Sorting
```

对应：

```text
Memory
Graph
Scheduler
Cache
```

计算机组成原理

对应：

```text
CPU
Cache
Memory
GPU
Tensor
Bandwidth
```

操作系统
```

对应：

```text
Process
Thread
Scheduling
Memory Management
Synchronization
Deadlock
I/O
```

计算机网络

对应：

```text
TCP
UDP
QUIC
TLS
NAT
P2P
Congestion Control
```

---

28. 每个 Milestone 的统一验收模板

每个里程碑必须写：

```text
## Goal

## Concepts

## Required Knowledge

## Mini Experiments

## Implementation

## Tests
```

```
## Benchmark

## Known Problems

## What I Learned

## Git Commit

## Next Milestone
```

---

29. Git 工作流

推荐分支：

```text
main
develop
feature/*
```

Commit 示例：

```text
feat: add local memory ingestion
feat: implement graph retrieval
feat: add device heartbeat
feat: add synapse protocol
feat: implement task scheduler
perf: optimize tensor transfer
fix: recover disconnected node
test: add memory sync tests
docs: add inference benchmark
```

每完成一个小 Milestone 都提交。

---

30. 技术债规则

发现问题时不要直接“糊过去”。

建立：

```text
TECH-DEBT.md
```

记录：

```text
Issue
Impact
Temporary Solution
Proper Solution
Priority
```

优先级：

```text
P0 = 阻塞系统
P1 = 严重影响正确性
P2 = 性能 / 可维护性
P3 = 优化
```

---

31. Research Log

建立：

```text
docs/research/
```

每个研究问题单独一篇。

例如：

```text
why-quic.md
model-sharding.md
memory-decay.md
crdt-memory-sync.md
scheduler-design.md
```

tensor-transfer.md
```

统一模板：

```text
# Research Question

## Problem

## Hypothesis

## Background

## Experiment

## Result

## Analysis

## Conclusion

## Next Question
```

---

32. 最终毕业 / 求职材料

项目最终必须产出：

1. README

回答：

```text
What
Why
Architecture
Quick Start
Benchmark
Roadmap
```

2. Architecture Document
```

完整系统设计。

3. Protocol Document

Synapse Protocol。

4. Benchmark Report

性能测试。

5. Security Model

威胁模型和安全边界。

6. Research Notes

核心技术研究记录。

7. Demo

至少：

```text
Local AI
Distributed Inference
Memory Graph
Cross-device Sync
Autonomous Event
```

---

33. 最终验收标准

SynapseMatrix v1.0 至少必须做到：

基础

- [ ] Web Console 可运行
- [ ] Control Plane 可运行
- [ ] AI Service 可运行
- [ ] PostgreSQL 可运行
- [ ] Neo4j 可运行
- [ ] Redis 可运行

Memory

- [ ] PDF ingestion
- [ ] Embedding
- [ ] Vector retrieval
- [ ] Graph retrieval
- [ ] Temporal retrieval
- [ ] Activation
- [ ] Decay

Device

- [ ] Device registration
- [ ] Capability discovery
- [ ] Heartbeat
- [ ] Offline detection

Network

- [ ] P2P connection
- [ ] Authentication
- [ ] Synapse Protocol
- [ ] Retry
- [ ] Reconnect

Distributed Inference

- [ ] Model loading
- [ ] Model partition
- [ ] Tensor transmission
- [ ] Multi-node inference
- [ ] Scheduler
- [ ] Fallback

Sync

- [ ] Full sync
- [ ] Incremental sync
- [ ] Versioning
- [ ] Conflict detection
- [ ] Recovery

Event Runtime

```
- [ ] Event bus
- [ ] Trigger
- [ ] Planner
- [ ] Task execution
- [ ] Memory write-back

## Security

- [ ] Device identity
- [ ] Encrypted communication
- [ ] Permission
- [ ] Revocation
- [ ] Audit log

## Observability

- [ ] Logs
- [ ] Metrics
- [ ] Traces
- [ ] Benchmark dashboard

---

# 34. Definition of Done
```

一个功能只有在以下条件全部满足时，才算完成：

```text
代码完成
+
Unit Test
+
Integration Test
+
必要的 E2E
+
Benchmark
+
文档
+
Git Commit
```

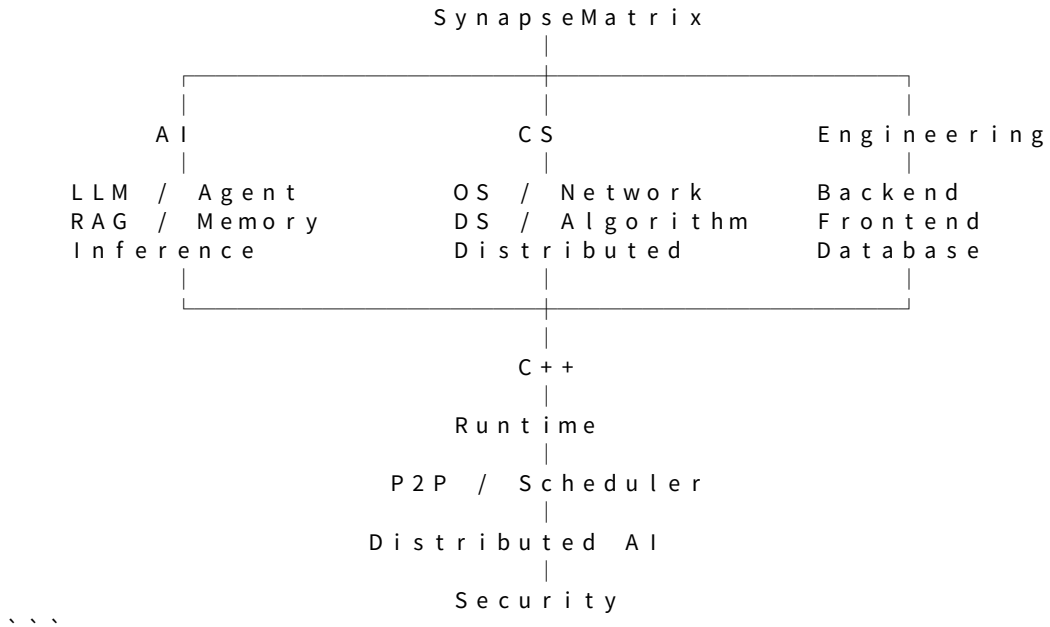
禁止：

```text
“能跑”
=
“完成”
```

---

35. 项目最终能力树

完成项目后，希望形成：



---

36. 最重要的学习原则

原则 1：项目驱动，不是视频驱动

不要问：

> 今天看什么？

应该问：

> 为了完成当前 Milestone，我缺什么？

原则 2：先理解，再调用框架

例如：

```text
先理解 TCP
  ↓
再理解 QUIC
  ↓
再使用 libp2p / MsQuic
```

原则 3：每学一个系统都做最小实验

例如学习调度：

```text
不要只看 Scheduling
  ↓
先写一个 Task Scheduler
  ↓
再加入 Priority
  ↓
再加入 Resource
```

原则 4：每个重大模块都要 Benchmark

不要只说：

> “优化了。”

一定要证明：

```text
Before
vs
After
```

原则 5：不懂的地方允许慢，但不能跳过

真正需要掌握：

```text
为什么
怎么工作
什么时候使用
代价是什么
失败会怎样
```

---

37. 未来研究扩展（V2+）

以下内容不属于 V1.0 必做，但可以作为研究方向：

```text
Federated LoRA
Model Quantization Research
Tensor Parallelism
Pipeline Parallelism
Speculative Decoding
Mixture of Experts
CRDT Semantic Merge
Encrypted Embedding
On-device Privacy Attack Research
WebRTC DataChannel
Multi-cluster Scheduling
NPU Runtime
Android Native Runtime
iOS Runtime
Edge GPU Scheduling
Local Multimodal Model
Personal Knowledge Graph Learning
```

---

38. 最终项目定位

正式对外介绍时使用：

> **SynapseMatrix 是一个端侧优先、P2P 协同、隐私保护的个人分布式认知计算系统原型。它将个人多设备组织为一个协同 AI 网络，通过分布式推理、认知记忆图谱、增量记忆同步与事件驱动任务运行，实现跨设备的个人智能计算。**

避免使用：

- “全球第一个”
- “完全零泄露”
- “绝对安全”
- “完全不依赖任何服务器”
- “比所有本地 AI 都强”

---

39. 12 个月结束时的自测题

如果以下问题仍然答不出来，说明项目虽然跑起来了，但知识还没有真正掌握。

C++

- RAII 为什么重要？
- Move Semantics 解决什么问题？
- atomic 与 mutex 有什么区别？
- Thread Pool 如何设计？
- 什么情况下会发生 data race？

OS

- Process 和 Thread 的区别是什么？
- Context Switch 发生了什么？
- Virtual Memory 是什么？
- Scheduler 如何工作？

Network

- TCP 与 UDP 的区别？
- TCP 为什么需要三次握手？
- QUIC 为什么建立在 UDP 上？
- RTT 如何影响分布式推理？

Distributed System

- 什么是一致性？
- Eventual Consistency 是什么？
- Version Vector 是什么？
- CRDT 为什么能够解决部分冲突？

AI

- Transformer 如何生成下一个 token？
- KV Cache 为什么能提升 Decode？
- Quantization 的代价是什么？
- 为什么分布式推理可能比本地更慢？

Memory

- Vector Retrieval 与 Graph Retrieval 的区别？
- Memory Activation 为什么需要衰减？
- 为什么 embedding 也需要隐私保护？

Scheduler

- 如何定义 Node Score？
- 如何处理节点突然离线？
- 如何避免某个节点长期过载？

Security

- TLS 在保护什么？
- 什么是 mTLS？
- Device Certificate 如何撤销？
- 什么是 Replay Attack？

---

40. 最终执行模式

从今天开始，学习顺序固定为：

```text
读取当前 Milestone
        ↓
理解目标
        ↓
列出知识缺口
        ↓
学习 CS / AI / Full Stack 理论
        ↓
完成 Mini Experiment
        ↓
接入 SynapseMatrix
        ↓
写 Unit Test
```

```
            ↓
写 Integration Test
            ↓
Benchmark
            ↓
记录 Research Log
            ↓
Git Commit
            ↓
Milestone 验收
            ↓
下一阶段
```

SynapseMatrix 不是“12 个月后必须做完的项目”。

它应该是：

> **你用 12 个月逐步获得计算机系统能力的主线。**

当一个模块超出当前能力时，不降低目标，而是把这个模块拆成更小的知识问题。

最终目标不是“做出了一个很牛的 Demo”，而是：

> **能够亲自解释、实现、测试、优化并维护这个系统。**
